

SINGAPORE-MIT ALLIANCE

Computational Engineering
SMA5232: Cluster and Grid Computing Technologies for Scientific Computing
COMPUTATIONAL LAB No. 3
19th July, 2006

MPI IMPLEMENTATIONS ON ELEMENTARY ALGORITHMS

Objective

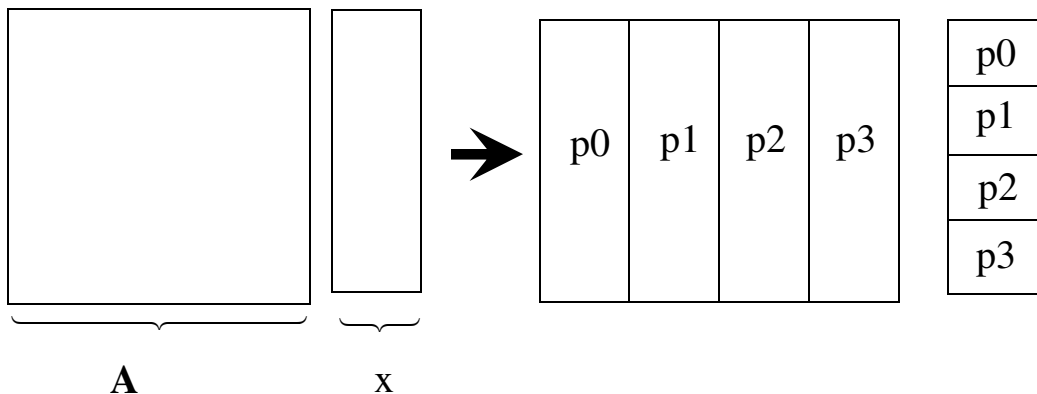
To experience MPI Library functions focusing on point to point and collective communications. To experience MPI Library functions focusing on grouping data for communications, communicators and topologies

Background knowledge

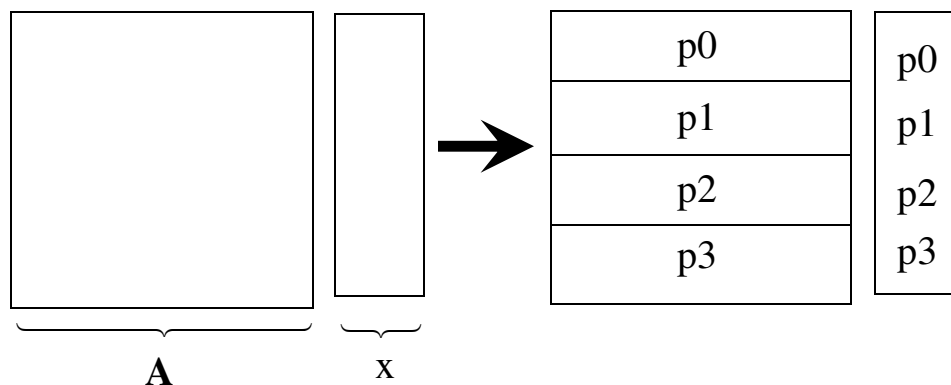
C/C++ languages, basic MPI functions and some ideas about communicator and topologies

1. Matrix-Vector Multiplication

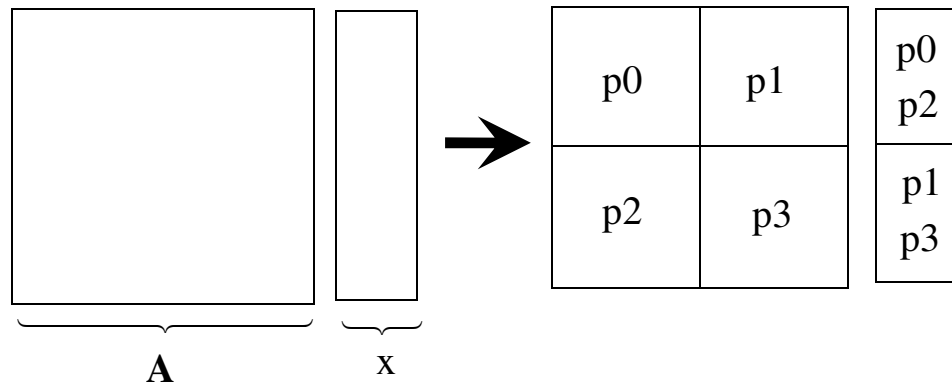
(a) Block-striped partitioning – column-wise block-striped partitioning



(b) Block-striped partitioning – row-wise block-striped partitioning



(c) Block-checkerboard partitioning



(d) Memory locations in C/C++

Recall that **MPI_Send**, **MPI_Recv**, **MPI_Bcast** and **MPI_Reduce** all have a count and a data-type parameter. These two parameters allow the user to group data items having the same basic type into a single message. In order to use this, the grouped data items must be stored in contiguous memory locations.

To understand how data are stored in the memory locations by using C/C++ language, the following code has been generated as follows:

```
#include <stdio>
#include <stdlib>
#include <iostream>
using std::cout;
using std::endl;
void understandDoubleArray(int N);
void understandEquivalentSingleArray(int N);

int main(int argc, char **argv)
{
    int N = 4;
    understandDoubleArray(N);
    cout << endl;
    understandEquivalentSingleArray(N);
    return 0;
}

void understandDoubleArray(int N)
{
    int **A;
    int i, j;

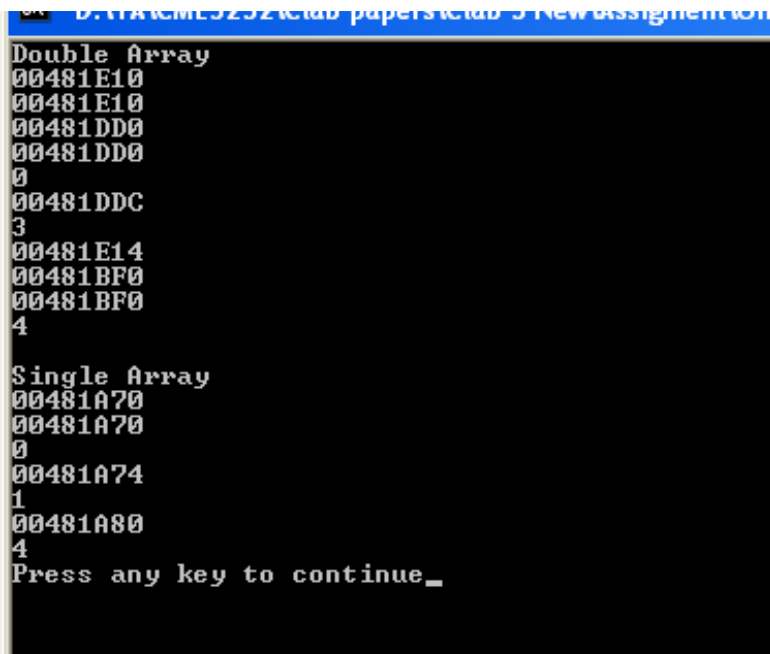
    A = new int*[N];
    for (i=0; i<N; i++)
        A[i] = new int[N];
    for (i=0; i<N; i++)
        for (j=0; j<N; j++)
            A[i][j] = i*N + j;
    cout << "Double Array" << endl;
```

```

        cout << A << endl;;
        cout << &(A[0]) << endl;
        cout << A[0] << endl;
        cout << &(A[0][0]) << endl;
        cout << A[0][0] << endl;
        cout << &(A[0][N-1]) << endl;
        cout << A[0][N-1] << endl;
        cout << &(A[1]) << endl;
        cout << A[1] << endl;
        cout << &(A[1][0]) << endl;
        cout << A[1][0] << endl;
    }
    void understandEquivalentSingleArray(int N)
    {
        int *A;
        int i, j;
        A = new int[N*N];
        for (i=0; i<N; i++)
            for (j=0; j<N; j++)
                A[i*N+j] = i*N + j;
        cout << "Single Array" << endl;
        cout << A << endl;
        cout << &(A[0]) << endl;
        cout << A[0] << endl;
        cout << &(A[1]) << endl;
        cout << A[1] << endl;
        cout << &(A[N]) << endl;
        cout << A[N] << endl;
    }
}

```

Result:



```

D:\TAKEME5252\lab papers\lab 5 New Assignment\com
Double Array
00481E10
00481E10
00481DD0
00481DD0
0
00481DDC
3
00481E14
00481BF0
00481BF0
4
Single Array
00481A70
00481A70
0
00481A74
1
00481A80
4
Press any key to continue_

```

It can be seen from the result above that C/C++ guarantees that single-array elements are stored in contiguous memory locations whereas the double-array elements are not stored in contiguous memory locations.

(e) What is the format of *matrix_vector_16.txt*?

```

16
1 1 0 4 3 2 3 2 1 0 4 2 0 0 0 4
4 2 3 0 4 3 4 2 1 4 0 2 3 1 2 1
4 0 0 3 0 1 1 1 2 1 0 3 3 2 2 2
4 2 0 1 4 3 4 1 4 4 0 1 4 2 1 1
1 4 0 1 2 1 3 0 1 4 1 3 3 0 4 1
0 3 4 1 2 4 4 2 2 3 2 4 3 4 0 2
3 0 0 0 0 1 3 4 1 3 1 0 4 0 2 2
1 4 3 4 4 3 3 4 2 4 3 2 3 0 4 3
1 2 2 2 2 2 0 2 1 4 4 2 3 3 4 0
0 2 4 1 2 0 1 2 0 0 3 2 4 1 4 3
3 2 3 2 3 3 0 3 0 3 3 1 1 0 2 2
2 2 1 4 3 3 0 0 2 0 3 4 2 2 1 1
3 1 1 1 2 1 4 3 0 3 0 0 4 0 0 1
3 3 4 3 3 4 4 4 1 4 2 2 4 3 2 2
1 0 1 0 2 0 1 3 0 2 1 3 0 3 4 2
4 0 0 0 4 4 3 3 3 0 3 2 1 2 0 4
0 4 2 3 4 0 3 3 2 3 1 1 1 2 0 1
  
```

Matrix Size = 16

16x16 matrix

16x1 vector

squareMatrixVectorGenerator.cpp is provided for you to generate matrices and vectors of different sizes.

2. Matrix-Matrix Multiplication Using Fox's Algorithm

(a) What is inside **serial_mat_mult.c**?

The following is the flow of **serial_mat_mult.c**.

- (i) Read in the matrix size n
- (ii) Read in the matrix **A** and **B**, and print them out
- (iii) Compute the matrix-matrix multiplication **AB**, store the result into matrix **C** and print **C** out

(b) What is inside **fox.c**?

The following is the flow of **fox.c**.

- (i) Setup the 2D Cartesian process grid.
- (ii) Process 0 read in the matrix size n and broadcast n to all the other processes.
- (iii) Read in matrix **A** and **B**, and print them out
- (iv) Use Fox's algorithm to compute the matrix-matrix multiplication, store the result into matrix **C** and print **C** out.

(c) What is Fox's algorithm?

The following is the flow of Fox's algorithm

- (i) Initialize the **local_C** to be all zero.
- (ii) Number of stages in the Fox's algorithm = the square root of the number of processes $\sqrt{p}$
- (iii) Loop through all the stages (Assume that $p = 16$, so number of stages is 4)

Stage 0

temp_A				local_B			
a ₀₀	a ₀₀	a ₀₀	a ₀₀	b ₀₀	b ₀₁	b ₀₂	b ₀₃
a ₁₁	a ₁₁	a ₁₁	a ₁₁	b ₁₀	b ₁₁	b ₁₂	b ₁₃
a ₂₂	a ₂₂	a ₂₂	a ₂₂	b ₂₀	b ₂₁	b ₂₂	b ₂₃
a ₃₃	a ₃₃	a ₃₃	a ₃₃	b ₃₀	b ₃₁	b ₃₂	b ₃₃

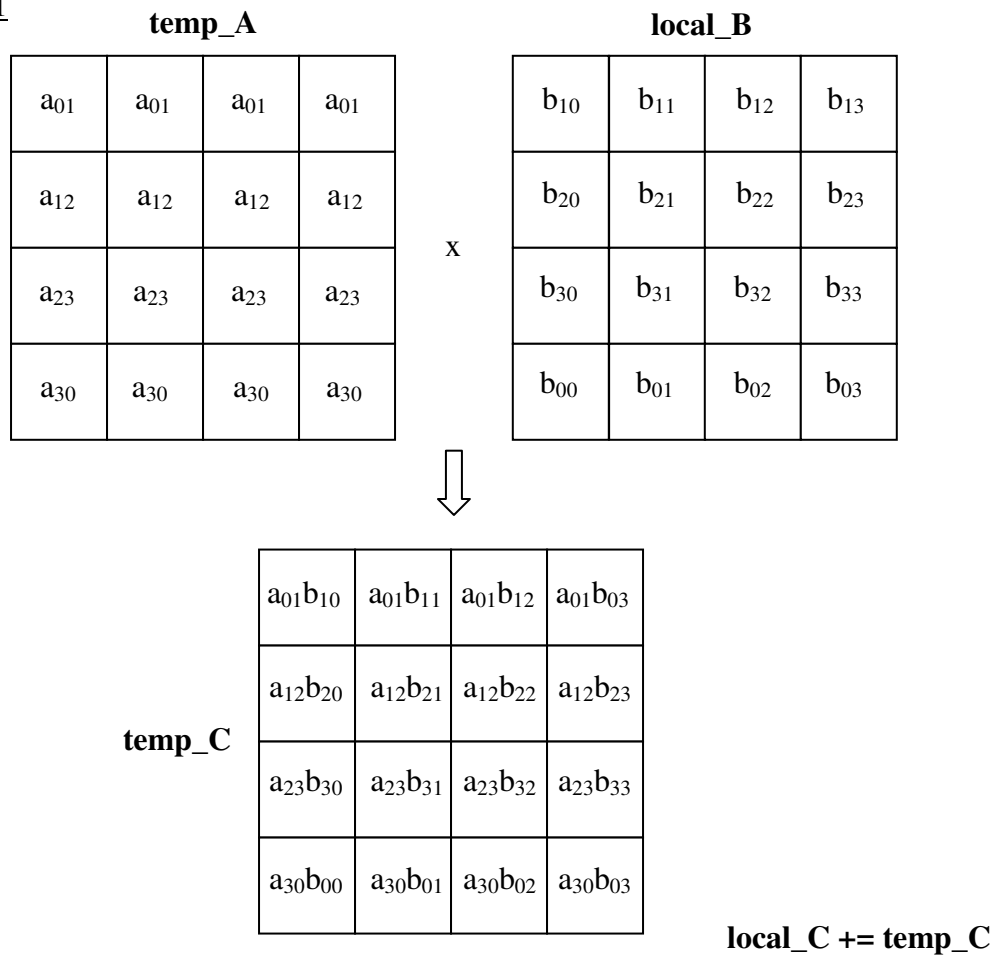
x



temp_C	a ₀₀ b ₀₀	a ₀₀ b ₀₁	a ₀₀ b ₀₂	a ₀₀ b ₀₃
	a ₁₁ b ₁₀	a ₁₁ b ₁₁	a ₁₁ b ₁₂	a ₁₁ b ₁₃
	a ₂₂ b ₂₀	a ₂₂ b ₂₁	a ₂₂ b ₂₂	a ₂₂ b ₂₃
	a ₃₃ b ₃₀	a ₃₃ b ₃₁	a ₃₃ b ₃₂	a ₃₃ b ₃₃

local_C += temp_C

Stage 1



So on and so forth for Stage 2 and Stage 3.

(d) Use of **squareMatrixMatrixGenerator.cpp**?

squareMatrixMatrixGenerator.cpp is provided for you to generate square matrices **A** and **B** of different sizes

```
$ gcc squareMatrixMatrixGenerator.cpp -o squareMatrixMatrixGenerator
$ ./squareMatrixMatrixGenerator
```

It will output a *matrix_matrix_<matrixSize>.txt* containing the matrices **A** and **B** information, and *matrix_matrix_<matrixSize>_result.txt* containing the result of the matrix-matrix multiplication. Open both files to see what is inside the files.